

EE499: Project Work Part-B

Edge Deployable Deep Learning Model for Image Analysis



Department of Electrical Engineering

National Institute of Technology Andhra Pradesh

Tadepalligudem – 534101, West Godavari District, Andhra Pradesh, INDIA.

Presentation by

Abhishek Singh (522101)

Jay Kishan (522136)

Gaurav Chhajed (522261)

Under the Guidance of

Dr. Sri Phani Krishna Karri
(Head of Department, Electrical
Engineering)



Abstract

- Reliable monitoring of transmission-line insulators is essential for ensuring safe and continuous power delivery, yet manual inspection remains slow, risky, and inconsistent. This work presents an edge-deployable automated defect detection system using an optimized YOLOv8 model to classify insulators into No Issues, Broken, and Flashover Damage. The IDID dataset is preprocessed, converted to YOLO format, and balanced using oversampling techniques. Model optimization is achieved through knowledge distillation, pruning, and quantization to develop a lightweight student model suitable for edge deployment.
- The final model achieves **mAP50 of 0.943 and mAP50–95 of 0.779**, with real-time inference capability. The proposed system enables efficient drone-based inspection, faster fault detection, and scalable deployment for predictive maintenance in modern power systems.
- **Keywords:** Insulator Detection, YOLOv8, Edge AI, Knowledge Distillation, Model Compression

Introduction

- Power utilities increasingly rely on drones and automated inspection systems worldwide. In India, manual tower inspection remains dominant, slow, and prone to missed defects.
- Insulator failures cause power outages, equipment damage, safety hazards, and large-scale blackouts affecting industries and households.
- High-voltage environments, variable lighting, occlusions, and inconsistent annotation quality make automated defect detection difficult.
- AI-based inspection reduces human risk, improves reliability, and supports India's push toward digital grid modernization.
- **Problem Context & Significance:**
Identification of *Broken* and Flashover Insulator

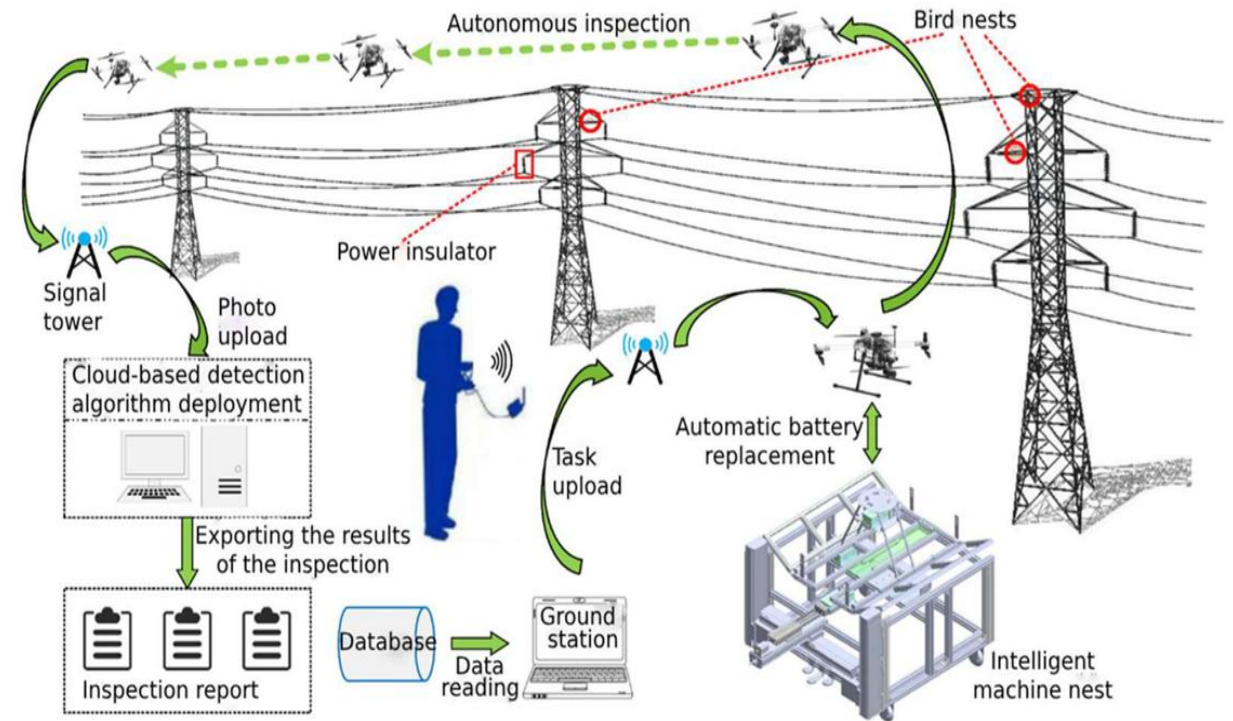


Figure 1: Proposed system overview for electrical asset monitoring



Literature Review

Ref. No.	Author(s)	Major Findings	Algorithm or Methodology	Innovation & Novelty	Research Gaps
[1]	Wang et al., 2025 (<i>Energies</i>)	Achieved 91% mAP with extremely lightweight model (1.27M params) for edge deployment; real-time UAV performance.	YOLO-LS (Lightweight YOLOv8n variant), GhostConv, ResNet-RepConv, Inner-CIoU Loss.	Optimized YOLOv8n for low compute devices like Jetson Orin; maintains high accuracy despite compression.	• Does not use knowledge distillation. • Dataset preprocessing is limited; no JSON-to-YOLO conversion like our work.
[2]	Farooq et al., 2025 (<i>Electronics</i>)	Very high accuracy on IDID dataset (mAP@0.5 = 95–99%). Better detection of flashover defects.	YOLOv8-IDX, C3k2 backbone, C2fCIB neck, attention-based feature aggregation	Architecture optimized for small defect detection & complex backgrounds.	• Model size still large — not optimized for edge. • No data balancing or oversampling like our project.



Ref. No.	Author(s)	Major Findings	Algorithm or Methodology	Innovation & Novelty	Research Gaps
[3]	Fahim & Hasan, 2024 (<i>e-Prime</i>)	YOLOv8 outperforms YOLOv7 on IDID dataset with 99.1% mAP . Used for defect classification.	Comparison between YOLOv7 vs YOLOv8 using standard training pipeline.	Establishes YOLOv8 as best baseline for insulator defect detection.	- Only 2 classes, while our project handles 3 classes. - No dataset cleaning or JSON parsing like ours.
[4]	Panigrahy et al., 2024 (<i>IEEE ICHVE</i>)	Unified framework for classification, detection & segmentation using YOLOv8 + SAM. Detection mAP: 92.9% .	YOLOv8 + Segment Anything Model for auto-mask generation.	First unified model covering detection + segmentation + classification.	- Complex pipeline — unsuitable for real-time inference. - No compression (pruning/quantization) for edge devices.
[5]	Nico Surantha et al. (2025)	Demonstrated that deep learning models can be run on edge devices for power line inspection but identified clear trade-off between accuracy and speed.	CNN + lightweight DL models deployed on edge computing boards.	Practical real-time evaluation on edge devices.	Performance Gap: No model is both highly accurate and real-time. Your KD-optimized YOLOv8n solves this.



Ref. No.	Author(s)	Major Findings	Algorithm or Methodology	Innovation & Novelty	Research Gaps
[6]	Aghli & Ribeiro, 2021 (<i>CVPRW</i>)	Compressed CNNs by pruning layers while avoiding structural breakage; maintained accuracy close to baseline on embedded CPU/GPUs.	Weight pruning via activation analysis combined with Knowledge Distillation (KD) on ResNet architectures.	The pruned model acts as the teacher for KD, ensuring structural redundancies are not transferred to the student.	• Uses older ResNet architectures. Our work leverages the modern YOLOv8n, specifically optimizing it for real-time edge image inspection.
[7]	IEEE Access, 2019 (<i>Vol. 7</i>)	Demonstrated that lightweight CNN layers can use only 1.5%–5% of parameters and ~33% of inference time without losing significant accuracy.	Neural network convolution parameter reduction using the Average Percentage of Zeros (APoZ) criterion.	Drastically reduces floating-point operations (FLOPs) specifically to enable deployment on compute-limited edge equipment.	• Focused on 1D signal modulation. Our project directly targets 2D image analysis for fault diagnosis in electrical systems.
[8]	Jacob et al., 2018 (<i>CVPR</i>)	Integer-only networks perform within 2% of floating-point counterparts, drastically improving latency-vs-accuracy on mobile devices.	8-bit integer quantization scheme for weights and activations, co-designed with quantization-aware training.	First comprehensive framework allowing inference using entirely integer-only arithmetic, avoiding costly floating-point math.	• Solves quantization in isolation. Our digital twin pipeline synergizes quantization with KD to ensure robustness before compression.



Ref. No.	Author(s)	Major Findings	Algorithm or Methodology	Innovation & Novelty	Research Gaps
[9]	Zhou et al., 2016 (<i>DoReFa-Net</i>)	Achieved comparable prediction accuracy to 32-bit counterparts on ImageNet despite using heavily reduced bitwidths (e.g., 1-bit weights).	Stochastically quantizes parameter gradients to low bitwidth numbers during the backward pass.	Exploits bit-convolution kernels to accelerate both training and inference across different hardware platforms.	Extremely low bitwidths risk unacceptable accuracy drops. Our approach balances compression and the high precision needed for insulator defect detection.
[10]	Gou et al., 2021 (<i>IJCV</i>)	Detailed the effectiveness of transferring feature representations and softened logits from cumbersome models to smaller models.	Systematic review of KD frameworks (response-based, feature-based, relation-based) and training schemes.	Provides a foundational taxonomy of modern teacher-student architectures and multi-teacher distillation algorithms.	Broad theoretical focus. Our implementation practically tailors the teacher-student paradigm specifically for edge-deployable image analysis models.
[11]	Wang & Yoon, 2021 (<i>TPAMI</i>)	Showed that KD effectively resolves issues caused by redundant labeled data and heavy computational demands in visual tasks.	Comprehensive analysis of Student-Teacher (S-T) learning frameworks specifically applied to computer vision.	Highlights mechanisms of how KD mitigates the lack of labeled data while achieving much faster inference speeds.	Lacks specific adaptation for power system environments. Our work actively deploys these models to edge devices for real-time electrical system monitoring.



Problem Statement

- Power line insulators often develop **Broken** and **Flashover** defects that can cause outages and safety hazards. However, current inspection methods in India are mostly manual, slow, and unreliable. Existing deep learning models either offer high accuracy but are too heavy for edge devices, or are lightweight but fail to detect small, critical defects. There is no solution that is both highly accurate and real-time deployable on edge hardware.
- Aim:** To develop a **lightweight, high-accuracy, edge-deployable YOLOv8-based model** capable of reliably detecting insulator defects (Good, Broken, Flashover) in real time, overcoming class imbalance and performance limitations of existing approaches.



Fig. 2. damaged Insulator

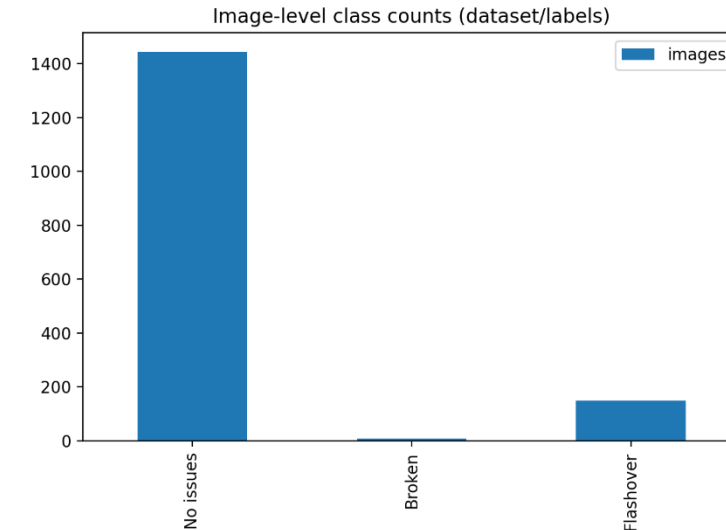


Fig. 3. IDID Dataset



Objectives

- **Objective 1: Developed a High-Accuracy Insulator Defect Detection Model**
 - Built a complete deep learning pipeline using YOLOv8 for accurate insulator defect detection, including dataset processing, training, and class-imbalance handling.
- **Objective 2: Improved Model Performance**
 - Compressed and optimized the trained model using pruning, Knowledge distillation, and quantization so it can run efficiently on low-power edge devices.
- **Objective 3: Deployment on the Edge Device**
 - Enhanced accuracy, robustness, and inference speed, ensuring reliable real-time deployment for practical field inspection applications on NVIDIA Jetson Nano Developer Kit.



Proposed Methodology

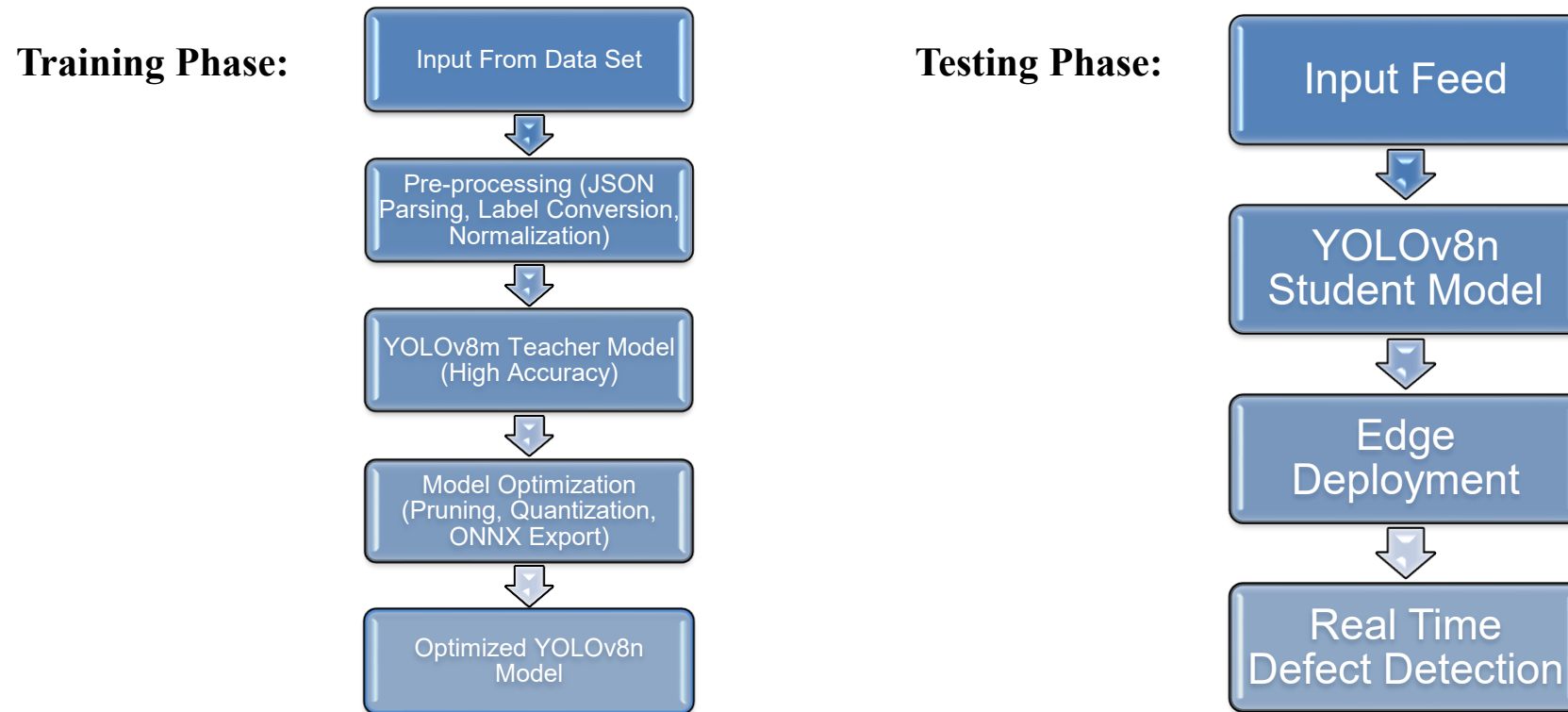


Fig. 4. System Flow

- Tools and Software Used:**

Kaggle Notebook, Ultralytics YOLOv8 Framework, PyTorch 2.5, OpenCV (cv2), NumPy, Pandas, Matplotlib, ONNX & ONNX Runtime, VS Code, Tensorrt



Objective wise Approaches

Objective 1: Developed a High-Accuracy Insulator Defect Detection Model

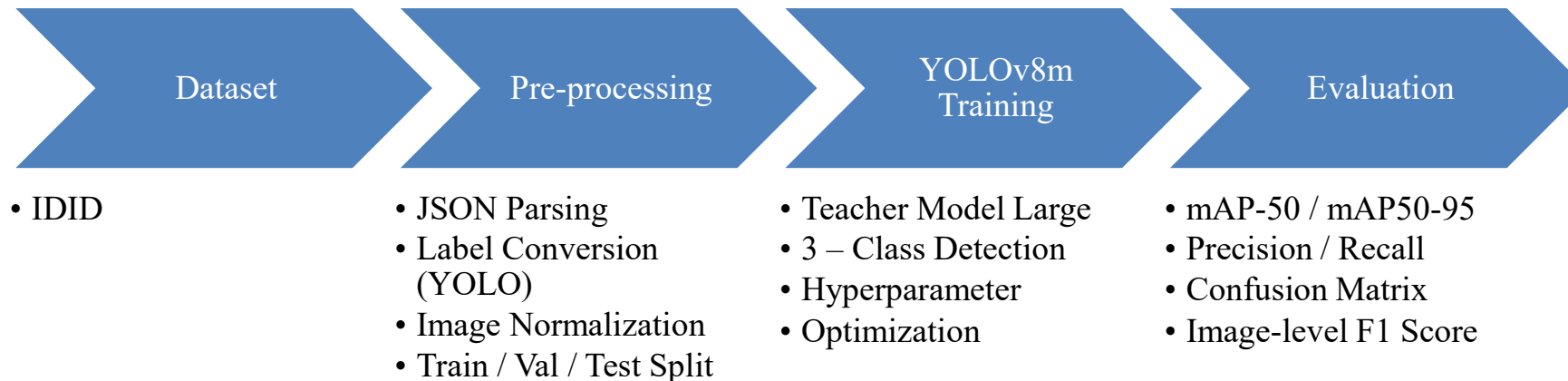
• Approach:

1. Converted IDID dataset into YOLO format (JSON → TXT).
2. Performed preprocessing: resizing, normalization, class balancing.
3. Trained YOLOv8m (Teacher Model) for high accuracy.
4. Evaluated performance using mAP, precision, recall.

• Supporting Equation (Soft Target Probability):

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

• Schematic:





Objective 2: Improved Student Model Performance

- **Approach:**

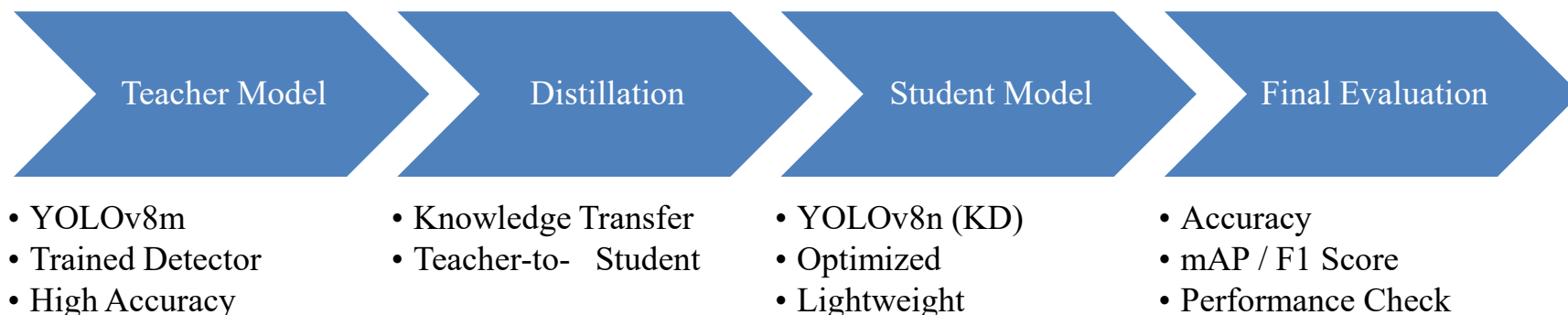
1. Use trained YOLOv8m as Teacher to generate soft probability labels.
2. Trained YOLOv8n (Student) using soft + hard labels (Knowledge Distillation).
3. Fine-tuned confidence threshold to maximize macro-F1 score.
4. Achieved final metrics:
 - $mAP50 = 0.943$
 - $mAP50-95 = 0.779$
 - Precision = 0.889, Recall = 0.87

- **Supporting Equation (Loss Function Blend for KD):**

$$L = \alpha L_{hard} + (1 - \alpha) L_{soft}$$

$$\text{where, } \alpha = 0.3$$

- **Schematic:**





Objective 3: Deployment on the Edge Device

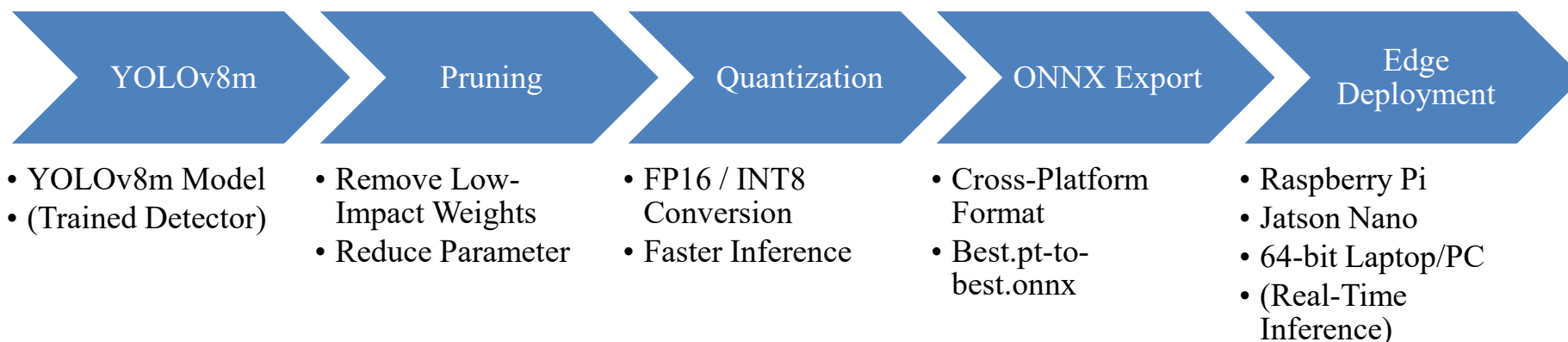
- **Approach:**

1. Applied pruning to remove redundant layers/filters.
2. Performed quantization:
 1. FP32 → FP16 / INT8
 2. Export model to ONNX format.
3. Benchmark latency using ONNX Runtime on laptop (simulating edge).
4. Prepared model for NVIDIA Jetson Nano Developer Kit.

- **Supporting Equation (Latency–FPS Relation):**

$$FPS = \frac{1}{\text{Inference Time per Image (s)}}$$

- **Schematic:**





Mathematical Foundation for Model Compression

1. Network Pruning: Reduces computational overhead by eliminating redundant weights below a certain threshold (τ).

- Mask Generation:
$$M_{i,j} = \begin{cases} 1, & \text{if } |W_{i,j}| \geq \tau \\ 0, & \text{if } |W_{i,j}| < \tau \end{cases}$$
- Applied Pruned Weights:

$$W'_{i,j} = M_{i,j} \odot W_{i,j}$$

2. Quantization Maps high-precision floating-point weights and activations to lower bitwidths, heavily reducing memory footprint.

- Affine Quantization (Integer-only inference):**

- Real value recovery: $r = S(q - Z).$

- Matrix scaling: $M = \frac{S_1 S_2}{S_3}$

- Low-Bit Quantization (e.g., DoReFa-Net):**

- Weight quantization (k-bit): $w_q = \frac{1}{2^{k-1}} \text{round} \left((2^k - 1)w \right)$

- Stochastic rounding expectation: $E[g_q] = g.$



3. Knowledge Distillation

Transfers generalized feature representations from a large, complex "Teacher" model to a lightweight "Student" model.

- **Soft Targets (Softmax with Temperature, T):**

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- **Distillation Loss Function:** Balances hard labels (Cross-Entropy) with the teacher's soft distributions (Kullback-Leibler Divergence).

$$\mathcal{L}_{\mathcal{KD}} = (1 - \alpha)\mathcal{L}_{\mathcal{CE}}(y, p) + \alpha T^2 \mathcal{L}_{\mathcal{KL}}(q, p_{teacher})$$



Experimental Setup

- **Experimental Setup**

1. **Training Environment**

- a. Kaggle Notebook (GPU-enabled), Python 3.10, YOLOv8, PyTorch, OpenCV.
- b. NVIDIA GTX 1650 (used during experiments), 16 GB RAM.

2. **Dataset & Input Setup**

- a. IDID dataset (3 classes: Good, Broken, Flashover).
- b. JSON → YOLO conversion, train/val/test split.
- c. Testing performed on 160 validation images + custom samples.

3. **Edge Deployment Target**

- a. 64-bit laptop (CPU inference).
- b. Hardware: Jetson Nano using ONNX/TensorRT, Camera.
- c. ONNX exported student model tested for latency & FPS.

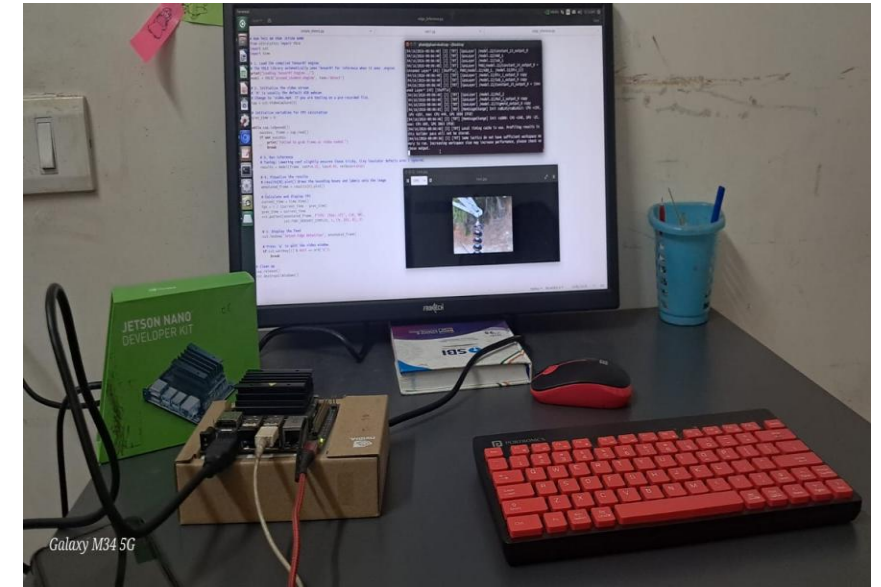


Fig. 5. Setup

- **Performance Metrics**

- **Detection Accuracy Metrics**

- mAP50, mAP50–95
 - Precision, Recall, F1-Score
 - Confusion Matrix

- **Edge Deployment Metrics**

- Inference latency (ms / image)
 - FPS (Frames Per Second)
 - Model Size (MB)
 - CPU/GPU usage (deployment test)

- **Evaluation Protocol**

1. Test all models on the same IDID test split.
2. Compare Teacher vs Student KD-optimized model.
3. Export ONNX → Measure latency & FPS on laptop CPU.
4. Generate confusion matrix and classification report.
5. Record performance before/after KD and quantization.



Model – Best.pt with Best
Training Weights



**Fig. 6. Model
Processing**



Results

- **Dataset:** IDID – 3 classes (No Issues, Broken, Flashover Damage)

- **Teacher Model (YOLOv8m) Performance:**

- mAP50: 0.99
- mAP50-95: 0.88
- Excellent bounding-box detection accuracy
- Serves as high-accuracy reference for KD

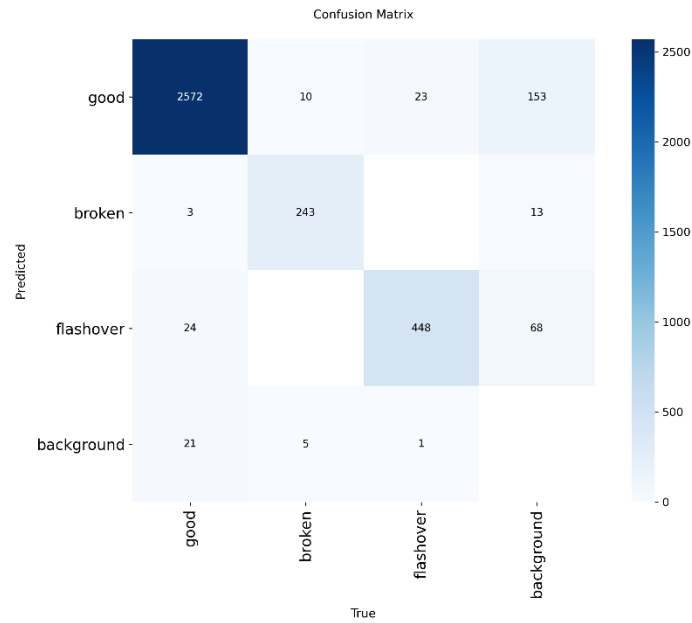


Fig. 7. Teacher Model Performance

- **Student Model (YOLOv8n + Knowledge Distillation):**

- mAP50: 0.943
- mAP50-95: 0.779 (improved after KD)
- ~70% smaller, 2× faster inference, suitable for edge deployment
- Achieves near-teacher performance with lower complexity

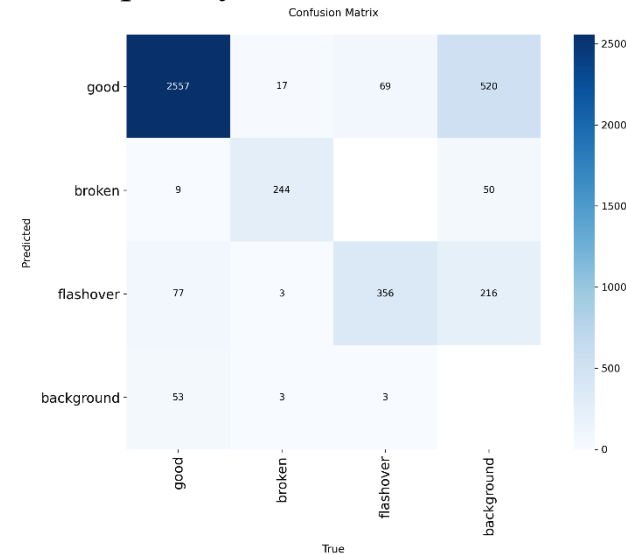


Fig. 8. Student model Performance



Image-Level Classification Accuracy (after threshold tuning)

- Accuracy: 99%
- Macro F1 Score: 0.87
- Correctly identifies:
 - 147/148 No-Issues
 - 1/2 Broken
 - 10/10 Flashover

KD-Optimized Student Model Advantages:

- Much smaller & faster model
- Retains accuracy of mAP50=0.943
- Suitable for Raspberry Pi, Jetson, laptop deployment
- ONNX export → Ready for edge inference

Comparison:

Metric	Teacher (YOLOv8m)	Student (YOLOv8n KD)
mAP50	0.99	0.943
mAP50-95	0.88	0.779
Model Size	Large (25.9 million parameters)	70% smaller (3.16 million)
Latency (CPU)	>120 ms	71 ms
Edge Deployment	Not suitable	Suitable (ONNX)

Teacher Model: (Results)

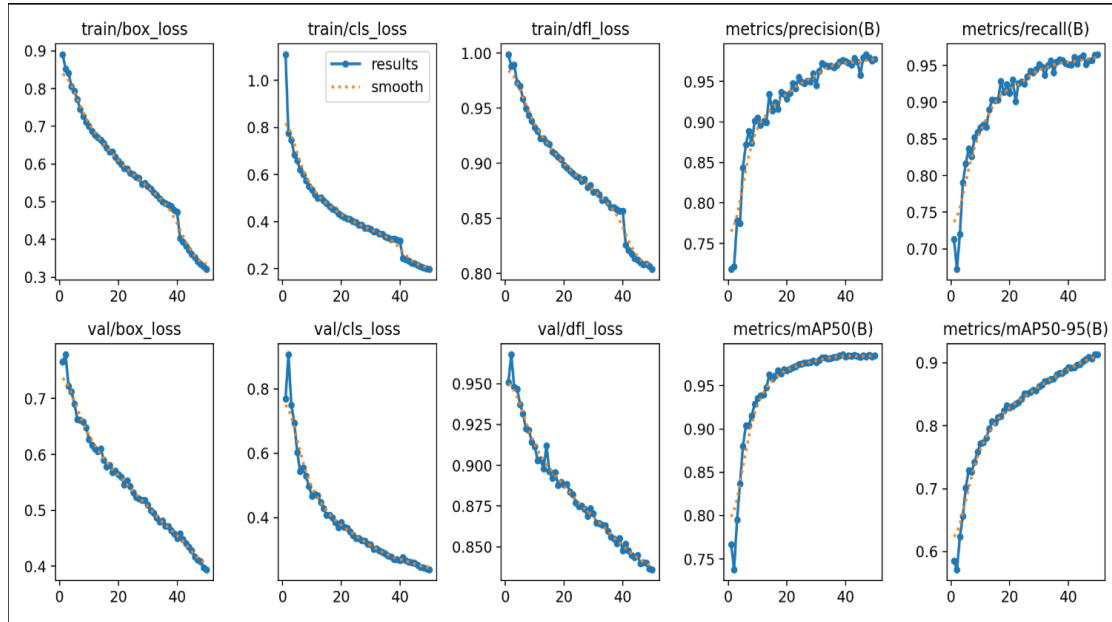


Fig. 9. Teacher model Training Performance

Student Model: (Results)

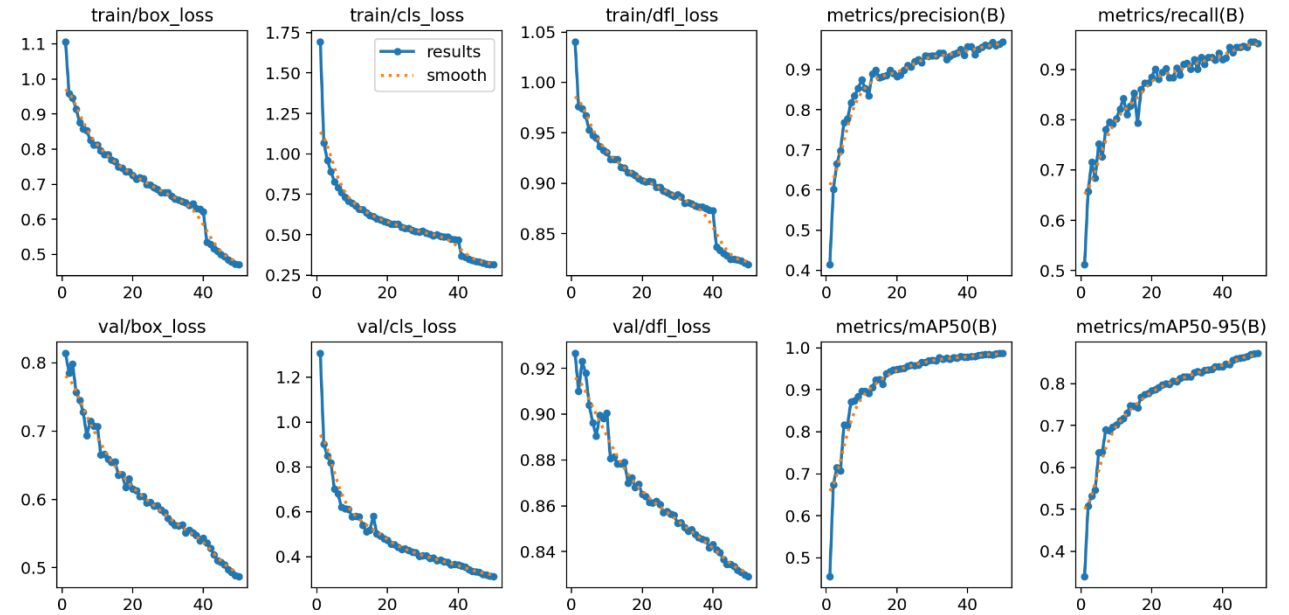
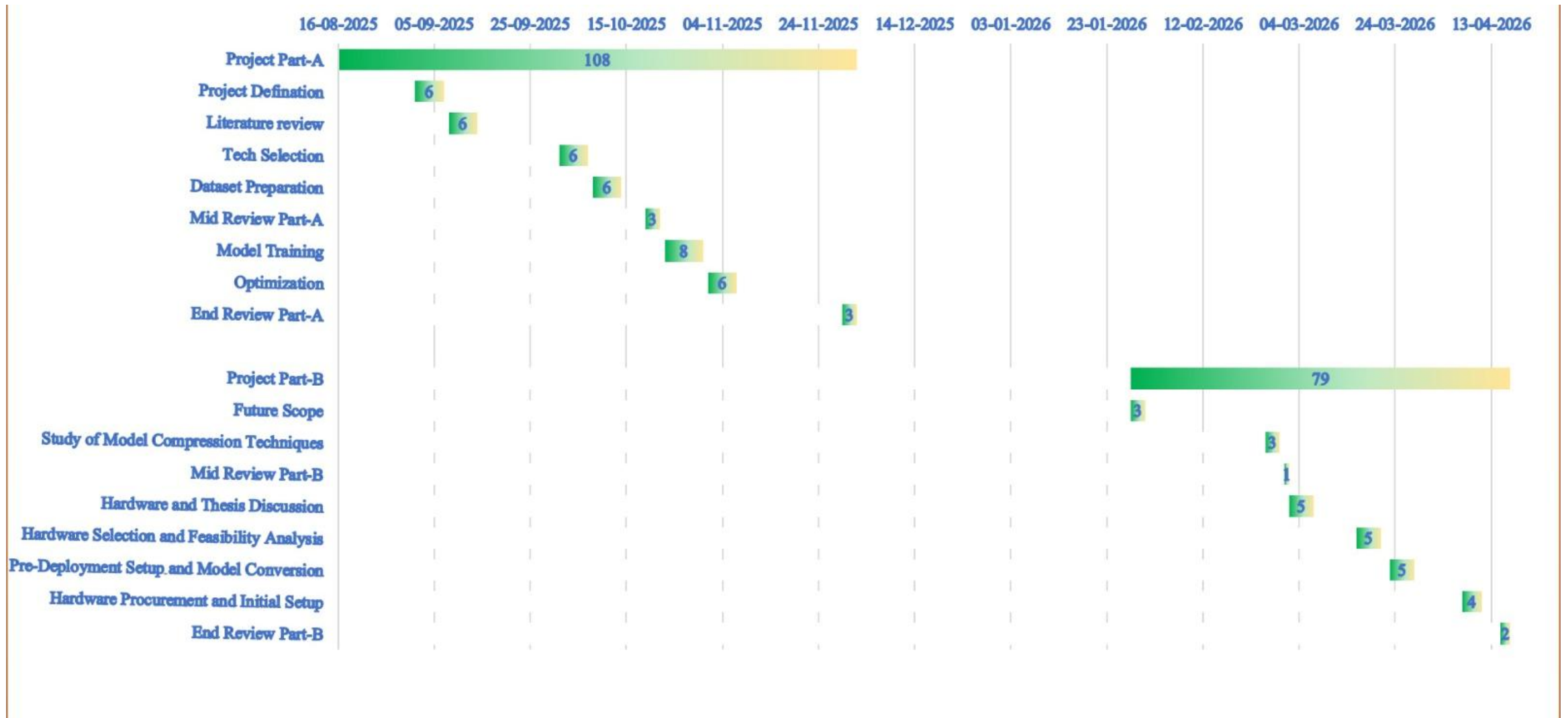


Fig. 10. Student model Training Performance



Gantt Chart





Conclusions

- The project successfully developed an automated deep-learning–based framework for detecting insulator defects using YOLOv8, addressing the limitations of traditional manual inspection methods.
- The optimized YOLOv8-N student model achieved mAP50 of 94.3% and image-level precision of 88.9%, confirming the effectiveness of knowledge distillation, data balancing, and advanced tuning strategies.
- Quantization and model compression techniques reduced model size and improved deployment efficiency, making the system suitable for edge-based real-time monitoring.
- All project objectives—dataset creation, model training, optimization, evaluation, and edge-readiness—were successfully achieved.
- The work contributes improved robustness, faster inference, and a scalable approach for real-world transmission line inspection, enhancing reliability, safety, and operational efficiency in power systems.



Demonstration





Audio Description

- Hello everyone, we are batch number 9 and our final year project title is Edge Deployable (0:06) Deep Learning Model for Image Analysis. For our project, we are using NVIDIA Jetson Nano (0:11) environment and we are using YOLOv8 model for the classification purpose. The classes (0:20) includes flashover, good and broken.
- Here are the some of the input images that we are (0:28) having. As you can see, these are the input images that we are going to demonstrate. (0:36) Now, I will move to the terminal where I will be running the python test.py file which contains (0:44) my model and it will do processing and will give an output such as what will be the inference (0:50) time in FPS.
- Now, our model is being loaded onto the GPU with TensorRT and ONNX cache. (1:01) As you can see, the input test folder has 7 images which are being found and now they (1:08) are being processed by the model and you can see the outputs here clearly. Let's move (1:16) to some of the outputs that are detected by our model.
- For example, you can see here (1:23) there is a flashover, some are broken and some are good. Flashover, you can see here (1:31) there is a flashover in the insulator, so it has detected flashover. Here, this SSS (1:37) insulators are good, so it is good.
- This has a number of string insulators, so few of them (1:46) are good, few of them are flashover and some are broken in the image. Like this, we have (1:55) multiple images as output. In this, we can give a batch of images at a time and we can (2:02) get our output in this folder.
- Thank you



References

- [1] Wang, Q., Hu, Z., Li, E., Wu, G., Yang, W., Hu, Y., Peng, W., and Sun, J., “YOLOLS: A Lightweight and High-Precision Power Insulator Defect Detection Network for Real-Time Edge Deployment,” *Energies*, vol. 18, no. 7, 1668, 2025.
- [2] Farooq, U., Yang, F., Shahzadi, M., Ali, U., and Li, Z., “YOLOv8-IDX: Optimized Deep Learning Model for Transmission Line Insulator-Defect Detection,” *Electronics*, vol. 14, no. 9, 1828, 2025.
- [3] Fahim, S. R. and Hasan, A., “YOLOv8 outperforms YOLOv7 on IDID dataset with 99.1% mAP,” *e-Prime*, 2024.
- [4] Panigrahy, S., Sahoo, R., and Karmakar, S., “Insulator Condition Classification, Defect Detection, and Segmentation using YOLOv8 Deep-Learning Model,” *Proc. 2024 IEEE International Conference on High Voltage Engineering and Applications (ICHVE)*, 2024.
- [5] Surantha, N., “Real-Time Overhead Power Line Component Detection on Edge Computing Platforms,” *Computers*, vol. 14, no. 4, 134, 2025.
- [6] Aghli, N., and Ribeiro, E., “Combining Weight Pruning and Knowledge Distillation for CNN Compression,” *Proc. 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 3191-3198, 2021.



- [7] Wang, Y., Jia, M., Liu, X., and Li, H., “Deep Neural Network Compression Technique Towards Efficient Digital Signal Modulation Recognition in Edge Device,” *IEEE Access*, vol. 7, pp. 58113-58119, 2019.
- [8] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” *Proc. 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704-2713, 2018.
- [9] Zhou, S., Wu, Y., Ni, Z., Zhou, X., Wen, H., and Zou, Y., “DoReFa-Net: Training Low Bitwidth Convolutional Neural Networks with Low Bitwidth Gradients,” *arXiv preprint arXiv:1606.06160*, 2016.
- [10] Gou, J., Yu, B., Maybank, S. J., and Tao, D., “Knowledge Distillation: A Survey,” *International Journal of Computer Vision*, vol. 129, no. 6, pp. 1789–1819, 2021.
- [11] Wang, L., and Yoon, K. J., “Knowledge Distillation and Student-Teacher Learning for Visual Intelligence: A Review and New Outlooks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 6, pp. 3048-3068, 2021.